

Санкт-Петербургский национальный исследовательский университет ИТМО
Мегафакультет трансляционных информационных технологий
Факультет информационных технологий и программирования

Лабораторная работа №3

по дисциплине «Проектирование баз данных»

Выполнил студент группы М3212
Молчанов Ростислав

Проверил
Папикян Сергей Седракович

IITMO

Санкт-Петербург
2026

Цель работы

Изучить механизмы повышения производительности запросов в PostgreSQL: обычные представления (VIEW), материализованные представления (MATERIALIZED VIEW) и индексы (INDEX). Оценить их влияние на скорость выполнения запросов с помощью EXPLAIN ANALYZE.

1. Создание объектов

1.1. Представления

Созданы два обычных и два материализованных представления:

- `v_order_details` / `mv_order_details` — детализация заказов (информация о пользователях);
- `v_product_sales` / `mv_product_sales` — агрегированная статистика продаж (суммарное количество единиц товара).

Пример создания:

```
CREATE VIEW v_order_details AS
SELECT o.id AS order_id, o.created_at, o.status, o.total, u.full_name
FROM orders o JOIN users u ON u.id = o.user_id;
```

```
CREATE MATERIALIZED VIEW mv_product_sales AS
SELECT p.id, p.name, SUM(oi.quantity) AS total_units_sold
FROM order_items oi JOIN products p ON p.id = oi.product_id
GROUP BY p.id, p.name;
```

1.2. Индексы

Созданы индексы для полей, участвующих в фильтрации:

```
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_orders_status_created_at ON orders(status, created_at);
CREATE INDEX idx_employees_status_hired_at ON employees(status, hired_at);
```

2. Анализ результатов EXPLAIN ANALYZE

2.1. Эффективность индексов

После создания индексов план запросов по точному совпадению `email = ...` изменился с `Seq Scan` на `Index Scan`. Для составных условий (`status = ... AND created_at >= ...`) оптимизатор стал использовать `Bitmap Index Scan` с последующим `Bitmap Heap Scan`, что ускорило выборку набора строк.

2.2. Случаи, когда индекс не используется

Даже при наличии индекса возможен `Seq Scan` в двух типичных ситуациях:

1. **Низкая селективность.** Запрос `WHERE status IN ('paid', 'created', ..., 'shipped')` выбирает почти всю таблицу. Последовательное чтение оказывается дешевле случайного доступа по индексу.
2. **Функция на индексируемом поле.** Условие не может использовать обычный индекс `idx_users_email`. Требуется функциональный индекс.

2.3. Сравнение обычных и материализованных представлений

- `VIEW` перевычисляет запрос при каждом обращении, включая `JOIN` и агрегацию. План выполнения сложный и тяжелый.
- `MATERIALIZED VIEW` хранит предвычисленный результат. Чтение из него значительно быстрее для повторяющихся аналитических запросов.

В работе это подтвердилось на примере `mv_order_details` и `mv_product_sales`: время выполнения запросов к материализованным представлениям оказалось на порядок ниже, чем к обычным.

Вывод

В ходе работы экспериментально подтверждено:

- Индексы эффективны для точечных и селективных запросов, но бесполезны (или вредны) при низкой селективности или применении функций к индексируемому полю.
- Материализованные представления обеспечивают существенный выигрыш в производительности по сравнению с обычными за счет хранения заранее агрегированных данных.
- Для обоснованного выбора метода оптимизации необходимо анализировать план выполнения через `EXPLAIN ANALYZE`.